ML-KEM Lab : cahier des charges complet pour Replit

À coller tel quel dans Replit Agent. Site pédagogique pour un exposé : simple à construire, beau, propre, interactif, avec de vraies valeurs ML-KEM.

0. Consignes à l'agent (à lire en premier)

Tu es un développeur web senior. Construis le site « ML-KEM Lab » décrit ci-dessous. C'est un projet d'exposé : je veux un site **fonctionnel, beau, propre, dynamique et interactif**, pas une architecture compliquée. Fais simple, mais exact.

Règles absolues :

1. **Vraies valeurs** : toutes les clés, tous les secrets et tous les ciphertexts affichés viennent de vrais calculs ML-KEM (FIPS 203). Aucune valeur inventée. Une illustration non calculée doit porter l'étiquette « Illustration ».
2. **Ne réécris jamais ML-KEM toi-même** : utilise la bibliothèque `@noble/post-quantum`. Pour SHA3 et SHAKE, utilise `@noble/hashes`. Pour HKDF et AES-GCM, utilise Web Crypto (`crypto.subtle`), natif au navigateur.
3. **Aucun backend** : tout se passe dans le navigateur. Aucune clé n'est envoyée sur un réseau ni écrite dans la console.
4. **Aucune erreur de mise en forme** : respecte la checklist visuelle de la section 4, c'est une exigence au même titre que le fonctionnement.
5. **Ne rajoute aucune fonctionnalité** en dehors de ce document, sauf le petit bouton optionnel « ciphertext altéré » décrit à la fin de la section 12.
6. **Tout le texte du site est en français**, avec les textes exacts donnés ici.

1. Objectif du projet

Créer une application web éducative qui fait comprendre visuellement le fonctionnement de **ML-KEM** (Module-Lattice-Based Key-Encapsulation Mechanism), standardisé par le NIST dans **FIPS 203** et issu de CRYSTALS-Kyber.

La simulation met en scène trois personnages :

- **Alice** : génère une paire de clés et veut établir un secret partagé avec Bob.
- **Bob** : utilise la clé publique d'Alice pour encapsuler un secret partagé.
- **Oscar** : espion qui intercepte les échanges et tente de retrouver les informations confidentielles.

L'application ne doit pas être un simple site documentaire : elle **exécute réellement** les opérations cryptographiques et affiche leurs résultats. C'est une simulation pédagogique locale à la session de l'utilisateur, qui ne prétend pas être un système de communication sécurisé prêt pour la production (le dire dans le pied de page).

2. Stack technique (choix tranchés)

Élément	Choix	Pourquoi
Outil de build	**Vite**	Démarre vite sur Replit, configuration minimale
Interface	**React 18, en JavaScript** (pas de TypeScript)	Composants réutilisables, moins d'erreurs
Styles	**CSS natif** avec variables CSS (pas de Tailwind, pas de bibliothèque d'UI ni d'animation)	Contrôle total de la palette et du rendu
ML-KEM	`@noble/post-quantum` (version exacte épinglée dans `package.json`)	Bibliothèque fiable, maintenue, conforme FIPS 203 (512, 768, 1024)
SHA3 / SHAKE	`@noble/hashes`	Déjà utilisée par noble, pour afficher les valeurs intermédiaires
HKDF et AES-GCM	**Web Crypto** (`crypto.subtle`)	Natif, standard, aucune dépendance
Tests	**Vitest** (quelques tests, voir section 12)	Intégré à Vite

Configuration Replit :

- `vite.config.js` : `server: { host: '0.0.0.0', port: 5173, allowedHosts: true }`.
- `.replit` : commande d'exécution `npm run dev`, port 5173 exposé. Déploiement en mode **Static** avec dossier de publication `dist` après `npm run build`.
- Polices : pile système uniquement (aucune police externe, aucun CDN).
- Vérifie à l'installation le chemin d'import exact de noble (`@noble/post-quantum/ml-kem.js` ou `/ml-kem`) en lisant le README de la version installée, et adapte les imports.

Point de vigilance Replit : `crypto.subtle` n'existe qu'en HTTPS ou en localhost (c'est le cas sur Replit). Si `crypto.subtle` est absent, affiche un message d'erreur clair. Le bouton « Copier » doit avoir un repli (`document.execCommand('copy')` via un `textarea` caché) car le presse-papiers peut être bloqué dans l'aperçu Replit.

3. Identité visuelle

Palette (exclusive)

Couleur	Code	Usage
Vert forêt	`#285943`	Boutons principaux, étapes importantes, opérations réussies
Gris foncé	`#292D32`	Textes et éléments secondaires
Blanc	`#FFFFFF`	Cartes de contenu
Vert très clair	`#EAF2EC`	Fond général du site

Seules ces quatre couleurs sont autorisées. Pour les bordures discrètes, les ombres et les états désactivés, utilise ces mêmes couleurs avec de la transparence (par exemple `rgba(40,89,67,0.15)` pour une bordure). **Pas de rouge, pas de bleu, pas d'orange.** Un échec d'Oscar se montre avec le gris foncé, un trait en pointillés et une icône (cadenas, croix), pas avec une couleur d'alerte.

Définis les couleurs une seule fois dans `:root` (`--vert` , `--gris` , `--blanc` , `--fond`) et utilise-les partout.

Style

Moderne, professionnel, épuré et facile à utiliser. Privilégier :

- Une excellente lisibilité.
- Des espaces bien équilibrés.
- Des cartes avec des bordures discrètes.
- Des animations douces et fluides.
- Des schémas clairs et bien organisés.
- Une hiérarchie visuelle évidente.
- Une interface responsive : ordinateur en priorité, téléphone pleinement utilisable.

À éviter : les effets visuels excessifs, les dégradés inutiles, les couleurs multiples, les interfaces surchargées.

Personnages et photos

Je fournirai moi-même les photos d'Alice, Bob et Oscar. Prévois :

- Les fichiers `public/avatars/alice.jpg`, `public/avatars/bob.jpg`, `public/avatars/oscar.jpg` (je les déposerai à ces emplacements).
- Un composant `Character` qui affiche la photo **dans un cadre circulaire** (`border-radius: 50%`, `object-fit: cover`, bordure vert forêt), avec le prénom dessous.
- Un **repli automatique** si l'image est absente (`onError`) : cercle vert forêt avec la lettre initiale en blanc. Le site doit donc fonctionner et rester beau avant que je dépose les photos.
- Oscar a une bordure en pointillés gris foncé pour le distinguer.

4. Checklist anti-erreurs de mise en forme (obligatoire)

C'est la partie la plus importante pour la qualité visuelle. Applique et vérifie chaque point :

1. `*, *::before, *::after { box-sizing: border-box; }` et `body { margin: 0; }`.
2. **Aucun défilement horizontal de la page**, de 360 px à 1440 px. Vérifie à 360, 390, 768, 1024 et 1440 px.
3. **Aucune carte ne se chevauche.** Utilise CSS Grid ou Flexbox, jamais de positionnement absolu pour la mise en page. Dans une grille, mets `min-width: 0` sur les enfants.
4. **Valeurs hexadécimales** : police monospace, `overflow-wrap: anywhere; word-break: break-all;`, zone avec hauteur maximale (`max-height: 180px; overflow: auto`) pour la valeur complète. Un texte long ne doit jamais dépasser de sa carte.
5. **Flèches entre les blocs** : fais-les en CSS simple (un trait vertical centré de 24 px avec une pointe en `::after`), ou un simple symbole centré. **Interdit** : lignes SVG superposées calculées à la main, connecteurs en position absolue, traits qui traversent le texte.
6. Une seule famille de police (pile système), une échelle de tailles cohérente (titres 28/22/18 px, texte 16 px, minimum 14 px), `line-height` entre 1.5 et 1.7.
7. Même rayon de bordure partout (12 px pour les cartes, 10 px pour les boutons), même épaisseur de bordure (1 px), même espacement (multiples de 8 px).
8. Boutons : hauteur minimale 44 px, texte jamais coupé, état survol, état focus visible (contour vert forêt), état désactivé lisible (opacité réduite) avec une info-bulle ou une phrase qui explique pourquoi.
9. Pas d'emoji dans l'interface. Pour les icônes, utilise de petits SVG inline simples (cadenas, clé, enveloppe, œil, copie, coche) dessinés en trait, couleur héritée.
10. Contraste suffisant : texte gris foncé sur blanc, texte blanc sur vert forêt.

11. Les animations n'utilisent que `opacity` et `transform` (durée 200 à 400 ms, `ease-out`). Respecte `prefers-reduced-motion` (animations désactivées).
12. Pas de saut de mise en page : réserve la hauteur des zones qui se remplissent, et n'insère pas un bloc en haut de la page qui pousse tout le reste.
13. Quand de nouveaux blocs apparaissent dans la phase, fais défiler doucement vers le dernier bloc révélé (`scrollIntoView({ behavior: 'smooth', block: 'nearest' })`).
14. Aucun lien bleu souligné par défaut, aucun style de navigateur par défaut visible sur les boutons ou champs.
15. Largeur maximale du contenu : 1100 px, centré, avec un padding horizontal de 16 px sur téléphone et 32 px sur ordinateur.

5. Structure générale du site

Le site commence par une **page d'accueil** présentant brièvement le projet, son objectif et le déroulement de l'expérience.

Texte d'accueil (exact) :

> « Bienvenue dans ML-KEM Lab. Découvrez comment deux personnes peuvent établir un secret partagé à travers un canal de communication non sécurisé, même en présence d'un espion. Suivez Alice et Bob dans une simulation interactive de ML-KEM, tandis qu'Oscar tente d'intercepter leurs échanges. »

Un **bouton principal** permet de commencer l'expérience.

Avant de commencer, l'utilisateur choisit le **jeu de paramètres** : ML-KEM-512, ML-KEM-768 ou ML-KEM-1024. **ML-KEM-768 est sélectionné par défaut.** Le choix modifie dynamiquement les tailles des clés, du ciphertext et les informations de niveau de sécurité, affichées dans un tableau (voir section 6).

La simulation se déroule en trois grandes phases, plus une démonstration finale :

1. Génération des clés par Alice.
2. Encapsulation par Bob.
3. Décapsulation par Alice.
4. Démonstration finale : chiffrement AES-GCM.

Oscar intervient entre les phases pour intercepter les données et tenter de compromettre la communication.

Une **barre de progression** indique la position de l'utilisateur dans le parcours. Elle comporte 4 segments (Clés, Encapsulation, Décapsulation, Canal chiffré), et de petits marqueurs « Oscar » entre les segments où il intervient.

6. Jeux de paramètres

Prends en charge les trois variantes officielles. Les tailles doivent être cohérentes dans toute l'application. Fais d'un seul fichier `src/crypto/params.js` la source unique de ces valeurs.

Paramètre	ML-KEM-512	ML-KEM-768	ML-KEM-1024
k	2	3	4
du	10	10	11
dv	4	4	5
Clé publique ek	800 octets	1184 octets	1568 octets
Clé privée dk	1632 octets	2400 octets	3168 octets
Ciphertext c	768 octets	1088 octets	1568 octets
Secret partagé	32 octets	32 octets	32 octets
Catégorie NIST	1	3	5

Formules de vérification : ek = 384k + 32 ; dk = 768k + 96 ; c = 32 × (du × k + dv) ; K = 32. Le code doit comparer les tailles réelles obtenues (`.length`) à ce tableau, et signaler toute différence.

Le changement de paramètre en cours de simulation demande une confirmation, car il recommence la simulation.

7. Les phases en détail

Chaque phase commence par une **carte de personnage** : photo, message de rôle, bouton d'action.

Chaque opération est un **bloc d'opération** (composant `OperationBlock`) qui présente :

- le nom de l'opération,
- une explication accessible (2 à 3 phrases simples),
- les entrées,

- les sorties,
- les **valeurs réellement générées** (aperçu hexadécimal avec « Afficher tout »).

Les blocs apparaissent **un par un** (environ 600 ms entre chaque), reliés par des flèches CSS (voir checklist). Un bouton « Passer l'animation » affiche tout d'un coup. **Important : les calculs sont faits en entier d'abord (quelques millisecondes), puis les blocs sont seulement révélés progressivement.** L'animation ne calcule rien.

Le schéma respecte l'ordre réel de FIPS 203. N'invente aucune étape intermédiaire.

7.1 Phase 1 : génération des clés par Alice

Message de la carte d'Alice (exact) :

> « Vous êtes dans la peau d'Alice. Vous souhaitez établir une communication sécurisée avec Bob. Pour commencer, vous allez générer votre paire de clés grâce à ML-KEM. »

Bouton : **« Lancer la génération des clés »**, qui exécute réellement ML-KEM.KeyGen.

Blocs, dans l'ordre :

1. **Graines aléatoires d et z**, chacune de 32 octets, tirées avec `crypto.getRandomValues`. Sortie : d, z.
2. **K-PKE.KeyGen(d)** : à partir de d, calcul de (ρ, σ) = SHA3-512(d ‖ k), puis génération de la matrice publique à partir de ρ, des vecteurs secrets s et d'erreur e à partir de σ, et calcul de la clé interne t = A·s + e. Valeurs affichées : ρ et σ (calculés réellement avec SHA3-512), taille de ek_PKE et de dk_PKE.
3. **Construction de la clé publique ek** : ek = ek_PKE, qui se termine par ρ. Sortie : ek.
4. **Construction de la clé privée dk** : dk = dk_PKE ‖ ek ‖ SHA3-256(ek) ‖ z. Valeurs affichées : le condensat SHA3-256(ek) calculé réellement, et les tronçons de dk.

À la fin, **deux cartes distinctes**, côte à côte sur ordinateur, empilées sur téléphone, avec un espace régulier entre elles et sans jamais se chevaucher :

Clé publique ek

- Taille réelle en octets.
- Aperçu de la valeur hexadécimale.
- Possibilité d'afficher la valeur complète.
- Possibilité de copier la valeur.
- Indication claire (badge) : « Peut être transmise ».

Clé privée dk

- Taille réelle en octets.

- Aperçu de la valeur hexadécimale.
- Possibilité d'afficher la valeur complète.
- Possibilité de copier la valeur.
- Indication claire (badge avec cadenas) : « Doit rester confidentielle ».

Le bouton **« Envoyer la clé publique à Bob »** apparaît à la fin. **La clé privée n'est jamais transmise** (aucun bouton d'envoi pour dk).

7.2 Première intervention d'Oscar : interception de la clé publique

Animation (composant `TransferStage`) avec les trois personnages en ligne : **Alice à gauche, Oscar au centre, Bob à droite**. La clé publique ek est représentée par un paquet numérique ou une enveloppe qui se déplace d'Alice vers Bob. Oscar intercepte une **copie** du paquet pendant le transfert. Bob et Oscar disposent donc tous deux de la clé publique.

Explication affichée (exacte) :

> « La clé publique n'est pas secrète. Oscar peut l'intercepter et l'observer. Cependant, connaître cette clé ne lui permet pas de retrouver la clé privée d'Alice. »

7.3 Phase Oscar : tentative de récupération de la clé privée

Message de la carte d'Oscar (exact) :

> « Vous êtes maintenant dans la peau d'Oscar. Vous avez intercepté la clé publique d'Alice. Votre objectif est de retrouver sa clé privée afin de compromettre leurs futures communications. »

Bouton : **« Essayer de retrouver la clé privée d'Alice »**.

Au clic : courte animation symbolisant une tentative d'analyse (barre ou compteur d'essais qui défile), **étiquetée « Illustration »** puisqu'elle ne calcule pas la vraie attaque. Puis le résultat (exact) :

> « La récupération de la clé privée nécessiterait de résoudre un problème mathématique considéré comme difficile. Les attaques connues, y compris quantiques, ne permettent pas de retrouver cette clé de manière réaliste avec les paramètres sélectionnés. »

Ne pas prétendre qu'une attaque est mathématiquement impossible dans l'absolu : expliquer la **difficulté computationnelle**. Afficher aussi la catégorie NIST du jeu choisi.

Bouton : **« Laisser tomber et attendre Bob »**.

7.4 Phase 2 : encapsulation par Bob

Message de la carte de Bob (exact) :

> « Vous êtes maintenant dans la peau de Bob. Vous avez reçu la clé publique d'Alice. Vous allez l'utiliser pour générer un secret partagé que seule Alice pourra retrouver grâce à sa clé privée. »

Bouton : **« Lancer l'encapsulation »**, qui exécute réellement ML-KEM.Encaps(ek).

Blocs, dans l'ordre :

1. **Génération d'un message aléatoire m** de 32 octets.
2. **Calcul de SHA3-256(ek)** (valeur réelle).
3. **Calcul de (K ‖ r)** à partir de SHA3-512(m ‖ SHA3-256(ek)) (valeurs réelles : les 32 premiers octets sont K, les 32 suivants sont r).
4. **K-PKE.Encrypt(ek, m, r)**.
5. **Génération des valeurs aléatoires nécessaires à K-PKE.Encrypt** : dans FIPS 203, elles ne sont pas tirées au hasard, elles sont **dérivées de r** (via SHAKE256). Ce bloc est présenté comme un sous-bloc de l'étape 4 et le texte l'explique.
6. **Construction du ciphertext c = c1 ‖ c2** : c1 = les 32 × du × k premiers octets de c, c2 = le reste (tailles réelles et valeurs réelles).
7. **Obtention du secret partagé K**.

L'ordre réel de FIPS 203 calcule K à l'étape 3 (avec r) ; l'interface le « révèle » à la fin de la séquence et l'explique en une phrase.

Résultats, deux cartes :

Ciphertext c

- Valeur hexadécimale consultable.
- Taille réelle en octets.
- Possibilité de copier.

Secret partagé K

- Valeur de 32 octets.
- Valeur consultable.
- Possibilité de copier.
- Indication qu'il reste confidentiel.

Bob ne transmet pas K à Alice. Il transmet uniquement c. Bouton : **« Envoyer le ciphertext à Alice »**.

7.5 Deuxième intervention d'Oscar : interception du ciphertext

Animation de transfert : **Bob à droite, Oscar au centre, Alice à gauche** (les personnages gardent leur place, seul le paquet change de sens). Le ciphertext c se déplace de Bob vers Alice. Oscar intercepte une copie, Alice reçoit aussi le ciphertext. Explication : le ciphertext peut être observé par un espion sans que cela lui permette de retrouver automatiquement le secret partagé.

7.6 Deuxième tentative d'Oscar

Message (exact) :

> « Vous êtes de nouveau dans la peau d'Oscar. Vous avez intercepté le ciphertext envoyé par Bob. Vous connaissez également la clé publique d'Alice. Votre objectif est maintenant de retrouver le secret partagé. »

Bouton : **« Essayer de retrouver le secret partagé »**.

Au clic : animation de tentative d'analyse, puis **vraie action** : Oscar génère sa propre paire de clés et lance `Decaps(dk_Oscar, c)`. Il obtient un secret K_Oscar **différent** de K (c'est le vrai comportement de ML-KEM, aucun message d'erreur n'est produit, le rejet est implicite). Afficher K_Oscar à côté d'une mention « K : confidentiel » pour montrer que K_Oscar ≠ K (ne pas afficher K ici). Puis le texte (exact) :

> « Malgré les informations interceptées, Oscar ne parvient pas à retrouver le secret partagé. Les problèmes mathématiques sur lesquels repose ML-KEM rendent cette récupération computationnellement irréalisable avec les moyens connus pour les paramètres choisis. »

Bouton : **« Abandonner »**.

7.7 Phase 3 : décapsulation par Alice

Message (exact) :

> « Vous êtes de retour dans la peau d'Alice. Bob vous a envoyé le ciphertext. Vous allez maintenant utiliser votre clé privée pour retrouver le secret partagé. »

Bouton : **« Décapsuler le ciphertext »**, qui exécute réellement ML-KEM.Decaps(dk, c).

Blocs, dans l'ordre :

1. **Extraction des composants de dk** : dk_PKE = dk[0 : 384k], ek = dk[384k : 768k+32], SHA3-256(ek) = dk[768k+32 : 768k+64], z = dk[768k+64 : 768k+96] (tronçons réels).

2. **Décomposition du ciphertext c en c1 et c2.**
3. **Décodage et décompression des données.**
4. **Déchiffrement à l'aide de la clé privée.**
5. **Récupération du message m′.**
6. **Recalcul de (K′ ‖ r′)** avec SHA3-512(m′ ‖ SHA3-256(ek)).
7. **Réencapsulation interne** pour reconstruire un ciphertext de vérification c′.
8. **Comparaison** du ciphertext recalculé c′ avec le ciphertext reçu c.
9. **Si la vérification réussit, restitution de K′.**

Comportement réel de FIPS 203, à respecter et à expliquer : Alice calcule **toujours** aussi K̄ = SHAKE256(z ‖ c) (32 octets). Si c′ = c, le résultat est K′. Si c′ ≠ c, le résultat est K̄ à la place : c'est le **rejet implicite**, aucune erreur n'est signalée, mais le secret obtenu est inutilisable. Ne remplace pas ce comportement par une simple comparaison artificielle.

Ce que le site affiche pour les blocs 5 à 8 (voir section 8, « Valeurs intermédiaires ») : m′ et c′ sont déduits de façon vérifiable (m′ est confirmé car SHA3-512(m′ ‖ SHA3-256(ek)) redonne bien K ; c′ = c si la vérification réussit). Aucun chiffre inventé.

Explication à afficher (panneau repliable « Transformation de Fujisaki-Okamoto ») : cette transformation convertit un chiffrement de base (K-PKE, sûr contre les attaques passives, IND-CPA) en un mécanisme d'encapsulation sûr contre les attaques actives, notamment les ciphertexts choisis par l'attaquant (**IND-CCA2**). Pour cela, Alice rechiffre le message qu'elle a retrouvé et vérifie que le résultat est identique au ciphertext reçu ; en cas de différence, elle renvoie une valeur pseudo-aléatoire (rejet implicite) au lieu d'une erreur, ce qui ne révèle rien à l'attaquant.

Résultat final

Afficher **côte à côte** :

- Le secret partagé de Bob : **K**.
- Le secret partagé récupéré par Alice : **K′**.

Vérifier **automatiquement** que K = K′ (comparaison en temps constant). Si les valeurs sont identiques, afficher le message de réussite (exact) :

> « Secret partagé établi avec succès ! Alice et Bob possèdent maintenant le même secret partagé, sans l'avoir directement transmis sur le réseau. Oscar a pu observer les échanges, mais il n'a pas pu retrouver ce secret. »

7.8 Démonstration finale : chiffrement AES

But : montrer que ML-KEM sert à **établir un secret partagé** et non à chiffrer directement des messages.

Scène de communication entre Alice et Bob (deux photos, bulles de discussion). Alice écrit le message :

> « Bob, j'ai peur d'Oscar, je pense qu'il m'espionne. »

Elle clique sur **« Envoyer le message »**. Expliquer que le secret partagé obtenu grâce à ML-KEM sert à dériver une clé AES. Afficher progressivement (mêmes blocs animés) :

1. **Message original.**
2. **Dérivation d'une clé AES** à partir du secret partagé avec **HKDF-SHA-256** (IKM = K, sel = 32 octets à zéro, info = « ML-KEM-Lab AES-256-GCM », sortie = 32 octets). Afficher la clé dérivée.
3. **Chiffrement AES-GCM** (AES-256, IV aléatoire de 12 octets, tag de 128 bits). Afficher IV, ciphertext et tag.
4. **Transmission du ciphertext.**
5. **Interception du ciphertext par Oscar.**
6. **Tentative de lecture du message par Oscar, sans succès** : il essaie réellement de déchiffrer avec une clé dérivée de son K_Oscar, et le déchiffrement échoue réellement (afficher « Échec du déchiffrement » avec cadenas, sans détail technique).
7. **Déchiffrement par Bob** à l'aide de la même clé.

Bob reçoit finalement (et répond avec un nouvel IV) :

> « J'ai bien compris Alice. Ne t'inquiète pas, notre communication est protégée. »

La démonstration distingue clairement trois badges de rôle, toujours visibles :

- **ML-KEM** : établissement du secret partagé.
- **HKDF** : dérivation de la clé de chiffrement.
- **AES-GCM** : chiffrement authentifié des messages.

Fin : bouton « Recommencer l'expérience ».

8. Moteur cryptographique (simple et fidèle)

Crée un seul fichier `src/crypto/mlkem.js` qui contient toutes les fonctions suivantes. Les calculs ne dépendent d'aucun composant visuel.

```
// Paramètres : ml_kem512, ml_kem768, ml_kem1024 de @noble/post-quantum
// keyGen(paramSet)  -> { ek, dk, d, z, rho, sigma, h }
// encaps(paramSet, ek) -> { c, K, m, h, r, c1, c2 }
// decaps(paramSet, dk, c) -> { Kprime, parts, kbar, accepted }
```

Appels à la bibliothèque (à vérifier selon la version installée)

- Génération de clés : `ml_kem768.keygen(seed)` avec `seed = d || z` (64 octets). La bibliothèque renvoie `{ publicKey, secretKey }` : publicKey = ek, secretKey = dk. Fournir la graine soi-même permet d'afficher d et z.
- Encapsulation : `ml_kem768.encapsulate(publicKey, msg)` avec msg = m (32 octets aléatoires), si la version le permet. Renvoie `{ cipherText, sharedSecret }`.
- Décapsulation : `ml_kem768.decapsulate(cipherText, secretKey)`. Renvoie le secret partagé (32 octets).

Si `encapsulate` ne permet pas de fournir m dans la version installée, laisse la bibliothèque le tirer et n'affiche pas m comme « valeur réellement utilisée » : affiche plutôt la preuve de cohérence ci-dessous.

Valeurs intermédiaires (pour les schémas)

La bibliothèque ne donne que les valeurs finales. Les valeurs suivantes sont **réellement calculées** avec `@noble/hashes` ou extraites des sorties, sans réimplémenter ML-KEM :

Valeur affichée	Comment l'obtenir
(ρ, σ)	SHA3-512(d ‖ k) : ρ = 32 premiers octets, σ = 32 derniers. k est écrit sur un octet.
ρ dans ek	Les 32 derniers octets de ek (doivent être égaux au ρ calculé ci-dessus : c'est un contrôle)
SHA3-256(ek)	`sha3_256(ek)` ; doit être égal au tronçon correspondant de dk
Tronçons de dk	Découpage de dk par `slice` (section 7.7)
(K ‖ r)	`sha3_512(m || sha3_256(ek))` ; les 32 premiers octets doivent être égaux au K donné par la bibliothèque (contrôle de cohérence)
c1 et c2	`c.slice(0, 32*du*k)` et `c.slice(32*du*k)`
K̄	`shake256(z || c, { dkLen: 32 })`

Valeur affichée	Comment l'obtenir
Vérification en Decaps	Si K′ = K, alors m′ = m (confirmé car SHA3-512(m ‖ H(ek)) redonne K) et c′ = c

Si un contrôle de cohérence échoue, le site affiche un message d'erreur générique (« Une vérification interne a échoué, relancez la simulation ») et n'affiche pas de valeurs douteuses.

Limite assumée : les coefficients polynomiaux internes (matrice A, vecteurs s, e, t, NTT) ne sont pas affichés. Les blocs correspondants sont expliqués avec leurs entrées et leurs sorties réelles, mais sans le détail polynôme par polynôme. Cela garde le site simple et exact.

Fichier `src/crypto/channel.js`

```
// deriveAesKey(K) -> HKDF-SHA-256 (Web Crypto) -> clé AES-256 (32 octets)
// aesEncrypt(key, texte) -> { iv, ciphertext, tag }  (AES-GCM, IV 12 octets aléatoire)
// aesDecrypt(key, iv, ciphertext) -> texte, ou erreur générique
```

Utilise `crypto.subtle.deriveBits` avec HKDF pour récupérer les 32 octets (afin de pouvoir les afficher), puis `importKey` pour AES-GCM. Le tag de 128 bits est constitué des 16 derniers octets du résultat de `encrypt`.

9. Organisation du projet (simple)

```
mlkem-lab/
├─ index.html
├─ package.json
├─ vite.config.js
├─ .replit
├─ public/avatars/             (alice.jpg, bob.jpg, oscar.jpg : mes photos)
└─ src/
   ├─ main.jsx
   ├─ App.jsx                  (affiche l'écran selon l'étape courante)
   ├─ crypto/
   |  ├─ params.js             (tableau des 3 jeux de paramètres, source unique)
   |  ├─ mlkem.js              (keyGen, encaps, decaps + valeurs intermédiaires)
   |  ├─ channel.js            (HKDF + AES-GCM)
   |  └─ bytes.js              (hex, concat, copie, comparaison en temps constant)
```

```
├─ state/
│  ├─ steps.js               (liste des étapes et prérequis)
│  └─ SimulationContext.jsx  (état global + secrets)
├─ content/texts.js          (TOUS les textes français)
├─ components/               (voir tableau ci-dessous)
├─ screens/
│  ├─ Home.jsx   ParamSelect.jsx   AliceKeyGen.jsx   OscarInterceptEk.jsx
│  ├─ OscarAttemptSk.jsx   BobEncaps.jsx   OscarInterceptCt.jsx
│  └─ OscarAttemptSs.jsx   AliceDecaps.jsx   AesDemo.jsx
└─ styles/ (tokens.css, global.css, components.css)
```

Règle : le dossier `crypto/` n'importe aucun composant React. Les composants n'effectuent aucun calcul cryptographique : ils appellent les fonctions de `crypto/` via le contexte puis affichent les résultats.

Composants réutilisables

Composant	Rôle
`Character` / `CharacterCard`	Photo ronde (avec repli lettre), prénom, message de rôle, bouton d'action
`DataCard`	Carte de donnée : titre, taille en octets, aperçu hex, « Afficher tout », « Copier », badge « Peut être transmise » ou « Confidentielle »
`HexViewer`	Valeur hex en monospace avec hauteur maximale et défilement interne
`CopyButton`	Copie avec repli ; affiche « Copié » 1,5 s
`OperationBlock`	Un bloc d'opération : nom, explication, entrées, sorties, valeurs
`OperationFlow`	Liste de blocs reliés par flèches CSS, révélation progressive
`TransferStage`	Scène Alice / Oscar / Bob avec paquet animé
`ExplainDrawer`	Explication repliable au clic (Fujisaki-Okamoto, rejet implicite, MLWE, etc.)
`ActionButton`	Bouton primaire ou secondaire, désactivé avec raison affichée
`ProgressBar`	Barre de progression en 4 segments avec marqueurs Oscar

Composant	Rôle
`ResultPanel` / `CompareSecrets`	Résultats et validation K = K′
`ParamTable`	Tableau dynamique des tailles selon le jeu choisi

10. État de la simulation et navigation

Un seul contexte React (`SimulationContext`) avec :

- `paramSet` (défaut `ML-KEM-768`),
- `step` (étape courante) et `maxStep` (étape la plus avancée atteinte),
- les résultats de chaque phase (keygen, encaps, oscar1, oscar2, decaps, aes).

Étapes dans l'ordre : `home` , `params` , `alice-keygen` , `transfer-ek` , `oscar-sk` , `bob-encaps` , `transfer-ct` , `oscar-ss` , `alice-decaps` , `aes-demo` .

Règles :

1. **Retour en arrière sans corruption** : revenir à une étape précédente change uniquement l'étape affichée. Les résultats déjà calculés sont ré-affichés tels quels, **sans recalcul ni nouveau tirage aléatoire**.
2. **Boutons désactivés si les prérequis ne sont pas remplis** (par exemple l'encapsulation exige que la clé publique ait été envoyée), avec une phrase qui explique pourquoi.
3. **Navigation** : la barre de progression permet de cliquer sur les étapes déjà atteintes, jamais sur les suivantes.
4. **Bouton « Recommencer » toujours visible** dans l'en-tête : remet tout à zéro et efface les secrets en mémoire (`fill(0)` sur les tableaux d'octets).
5. **Aucune persistance** : ni localStorage, ni sessionStorage, ni cookie.
6. Les secrets (dk, K, K′, clé AES) sont gardés dans des `Uint8Array` ; ne les mets pas dans l'URL ni dans la console.

11. Animations

- Révélation des blocs : opacité 0 vers 1 et translation de 8 px vers 0, 300 ms, un bloc toutes les 600 ms environ.
- `TransferStage` : trois colonnes fixes (Alice 15 %, Oscar 50 %, Bob 85 %). Le paquet (petite enveloppe SVG) se déplace avec `transform: translateX(...)` d'une extrémité vers le centre, une **copie** du paquet descend vers Oscar, puis le paquet original continue jusqu'à la destination. Durée totale 3 à 4 secondes. Le sens s'inverse pour Bob vers Alice, les personnages ne changent pas de place.

- Animation de tentative d'Oscar : compteur d'essais qui défile dans une carte, étiqueté « Illustration », puis arrêt avec cadenas.
- Transition entre écrans : fondu de 250 ms.
- Aucun effet excessif, aucun défilement automatique brutal, aucune animation en boucle infinie sauf un très discret indicateur de chargement.

12. Exigences cryptographiques, sécurité et tests

Exigences

- Implémentation réelle et conforme à NIST FIPS 203 (via `@noble/post-quantum`).
- Aucune réimplémentation d'un algorithme cryptographique complexe.
- Fonctions cryptographiques réellement exécutées ; valeurs affichées issues de résultats réels.
- Données fictives uniquement pour des illustrations explicitement étiquetées.
- Les secrets et clés privées ne sont envoyés à aucun service externe et n'apparaissent pas dans les logs (pas de `console.log` de données cryptographiques).
- Gestion correcte des erreurs : messages généraux en français, sans détails internes sensibles ni trace technique.
- Documenter les dépendances cryptographiques dans un petit fichier `README.md` (noble post-quantum, noble hashes, Web Crypto).
- Aléa : uniquement `crypto.getRandomValues` (jamais `Math.random`).
- Pied de page : « Outil pédagogique : ne protège pas de vraies communications. »

Tests minimaux (Vitest, 6 tests suffisent)

1. Pour chacun des 3 jeux : les tailles réelles de ek, dk, c et K correspondent au tableau de la section 6.
2. Aller-retour : `K === K'` sur 50 exécutions pour chaque jeu.
3. Structure de dk : `dk = dk_PKE ‖ ek ‖ SHA3-256(ek) ‖ z` (le tronçon de hash vaut `sha3_256(ek)` et le tronçon final vaut z).
4. Les 32 derniers octets de ek valent ρ = les 32 premiers octets de SHA3-512(d ‖ k).
5. Les 32 premiers octets de SHA3-512(m ‖ SHA3-256(ek)) valent K.
6. Rejet implicite : si on inverse un bit de c, `Decaps` ne lève aucune erreur, renvoie K′ ≠ K, et K′ vaut SHAKE256(z ‖ c).

Optionnel mais recommandé : un test AES-GCM (chiffrement puis déchiffrement avec la bonne clé réussit, avec une autre clé échoue).

Option : ciphertext altéré (le seul ajout autorisé)

Dans l'écran de décapsulation, un petit bouton secondaire « Tester avec un ciphertext altéré » inverse un bit de c et relance `Decaps` sur ce c modifié. Il montre le rejet implicite réel : c′ ≠ c, résultat = K̄ ≠ K, sans erreur. Il n'écrase pas le résultat principal.

13. Plan de développement en 5 étapes

Réalise une étape à la fois, et vérifie son critère avant de passer à la suivante.

Étape	Contenu	Critère de validation
1. Socle	Projet Vite + React, thème (palette, composants de base), page d'accueil, choix du jeu de paramètres, `ParamTable`	L'application s'ouvre dans l'aperçu Replit, les 3 jeux changent les tailles, aucun défilement horizontal
2. Moteur	`params.js`, `mlkem.js`, `channel.js`, `bytes.js` + les 6 tests	Tous les tests passent pour les 3 jeux
3. Phases 1 à 3	`Character`, `DataCard`, `OperationBlock`, `OperationFlow`, `TransferStage`, écrans Alice/Bob/Oscar, barre de progression	Parcours complet sur ML-KEM-768 jusqu'à « K = K′ », cartes sans chevauchement de 360 à 1440 px
4. Démo AES	`AesDemo`, échec réel d'Oscar, réponse de Bob	La démo se déroule et le déchiffrement d'Oscar échoue réellement
5. Finition	Navigation arrière, désactivation des boutons, « Recommencer », mode mouvement réduit, relecture de la checklist de la section 4, README	Parcours complet sur chacun des 3 jeux, sur téléphone et ordinateur

Checklist finale avant livraison

- Parcours complet réussi sur ML-KEM-512, 768 et 1024.
- Les photos d'Alice, Bob et Oscar s'affichent en rond sans déformation (et le repli lettre fonctionne sans elles).
- Aucune couleur hors palette dans les fichiers CSS.
- Aucun chevauchement, aucun texte coupé, aucun défilement horizontal à 360, 390, 768, 1024 et 1440 px.
- Aucune requête réseau pendant la simulation (vérifier l'onglet Réseau).

- Tous les textes exacts de ce document sont présents.
- Les valeurs hex affichées correspondent aux vrais résultats et la copie fonctionne.
- `npm run build` réussit et le déploiement Static fonctionne.